

Generative Image Models

Part 2

In this part we will cover the latest breakthroughs in image generation: Diffusion and Stable Diffusion Models



Table of Contents

01

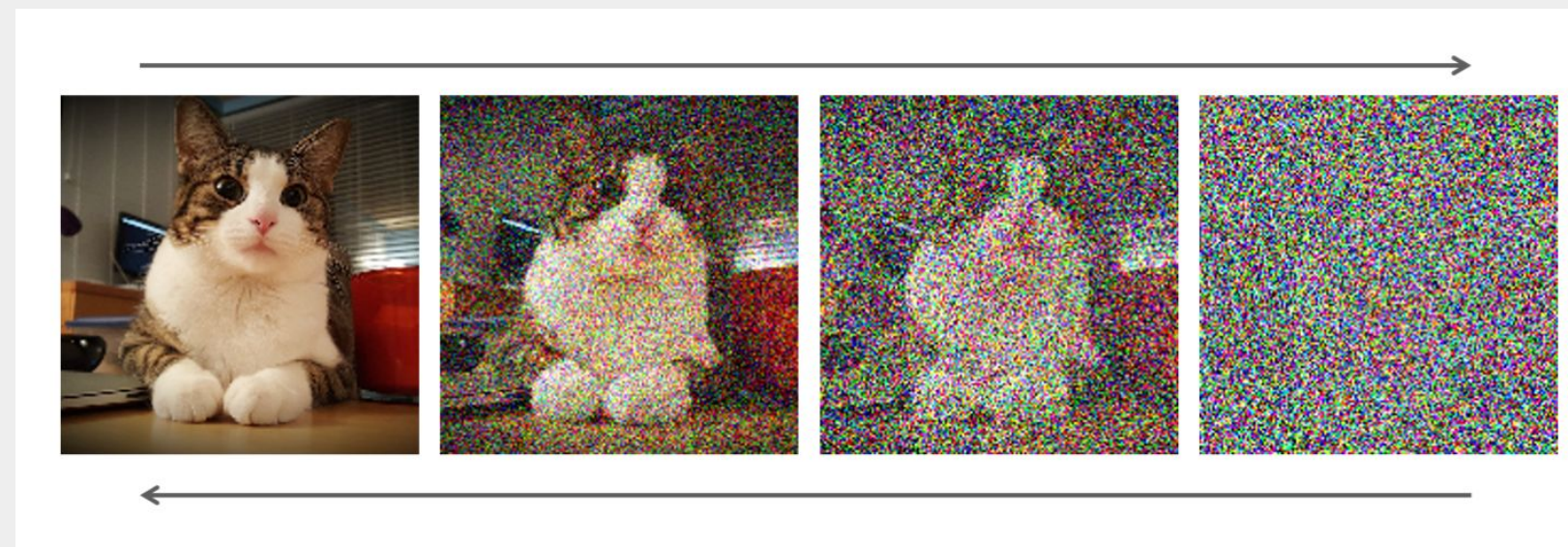
Diffusion Models

02

Vision Language Models

- Stable Diffusion
- CLIP

Diffusion Models



Diffusion Models

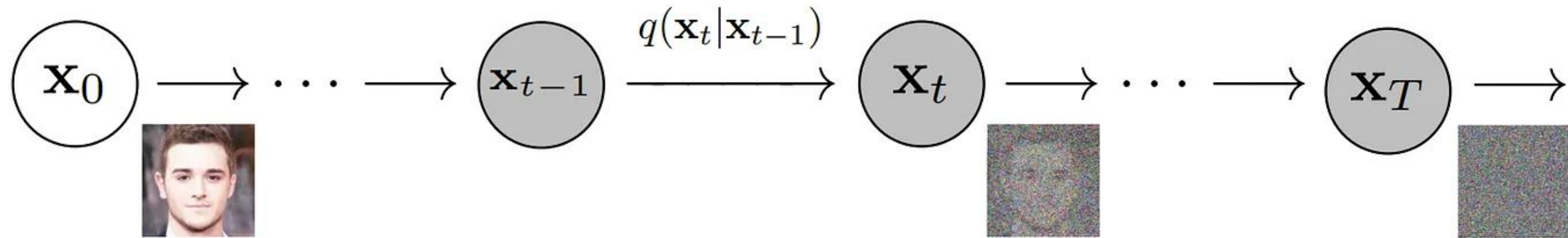
Diffusion Models are inspired by principles of thermodynamics, particularly the concept of diffusion processes. They work by destroying training data through the successive addition of **Gaussian** noise and then learning to recover the data by reversing this noising process.

Forward Process: Gradually add Gaussian noise to a clean image over many steps (e.g., 1000 steps) until it becomes pure random static.

Reverse Process (Denoising): Train a neural network (the U-Net) to predict the noise added at each step. By subtracting this predicted noise, the model slowly sculpts an image out of static.



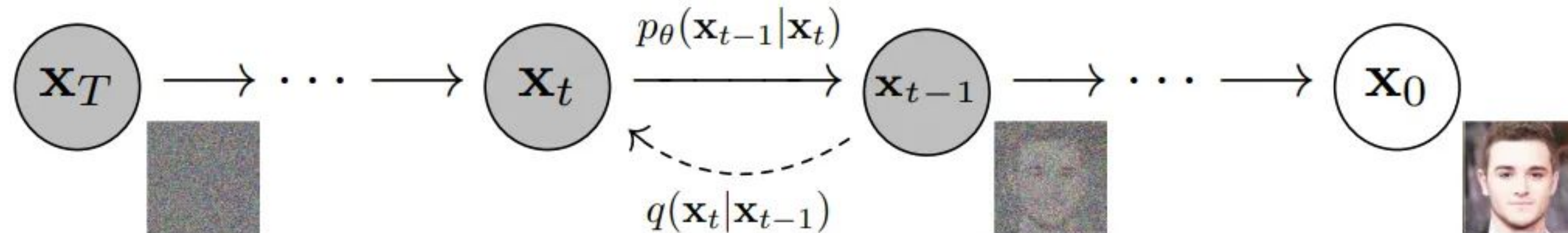
Adding Gaussian Noise to images



We split the process in T steps, in each step we add an amount of noise. The noising process is modelled as a **parameterized Markov chain** that incrementally adds noise to an image, leading it towards becoming pure Gaussian noise. The noise follows the Gaussian distribution and also each noise pixel introduced, has random values between black and white.

This process is executed by the noise scheduler that controls the amount of noise added to the data at each timestep during the forward diffusion process. The scheduler defines how the noise level evolves across timesteps, from nearly no noise at the beginning to almost complete noise at the final step.

Model training: Removing Noise from images

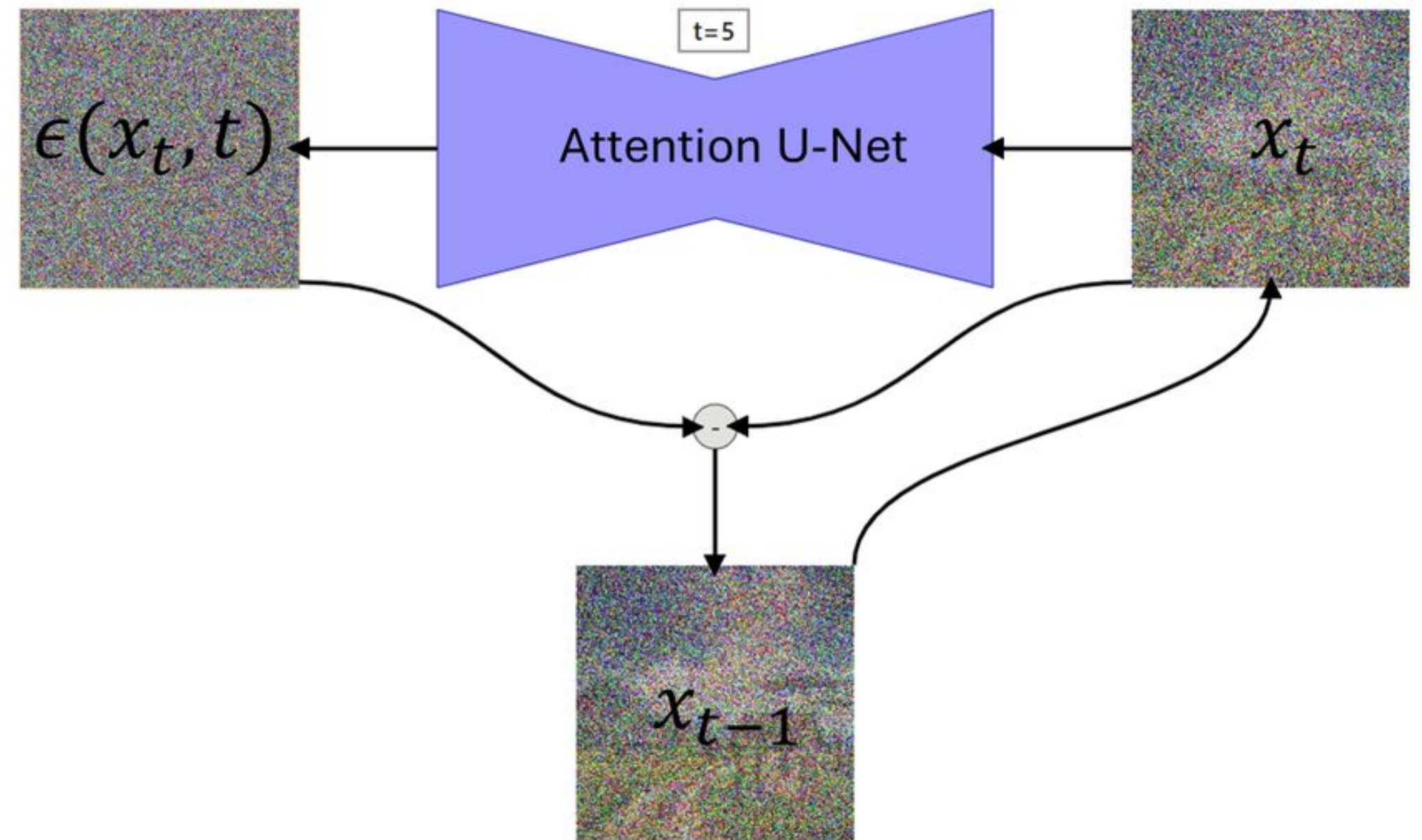


During training, the model is trained to predict the noise added to the data at each step, so it can subtract that noise and recover the original data.

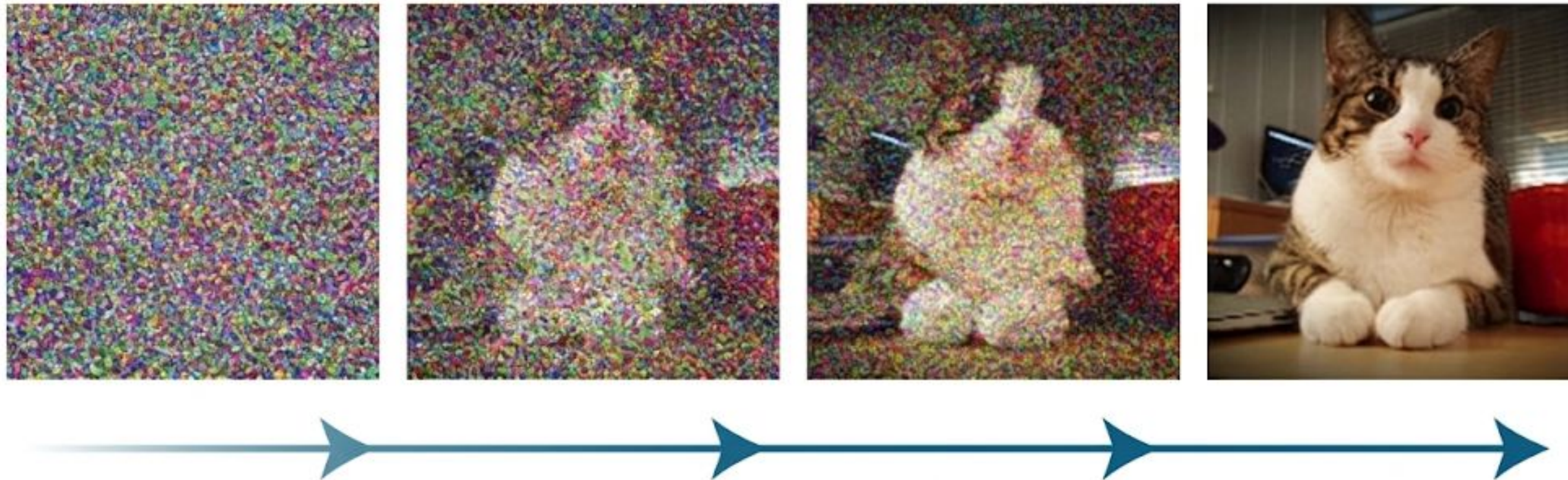
The model is trained using a loss function to minimize the difference between the **predicted noise** and the **actual noise added during the forward process**. The most common objective is the mean squared error (MSE) between the predicted noise and the actual noise.

Model training: Removing Noise from images

After the network predicts the total noise for the given $t=5$ to go the $t=0$, it adds back this noise to the image minus a fraction of that noise (producing $t=4$).



Reverse Diffusion



We can use the trained model to generate an image from noise. This process happens one step at a time, at exactly the same t steps happened during the training process.

Slow Sampling

Generating a single image requires hundreds of sequential evaluations, making it far slower than GANs.

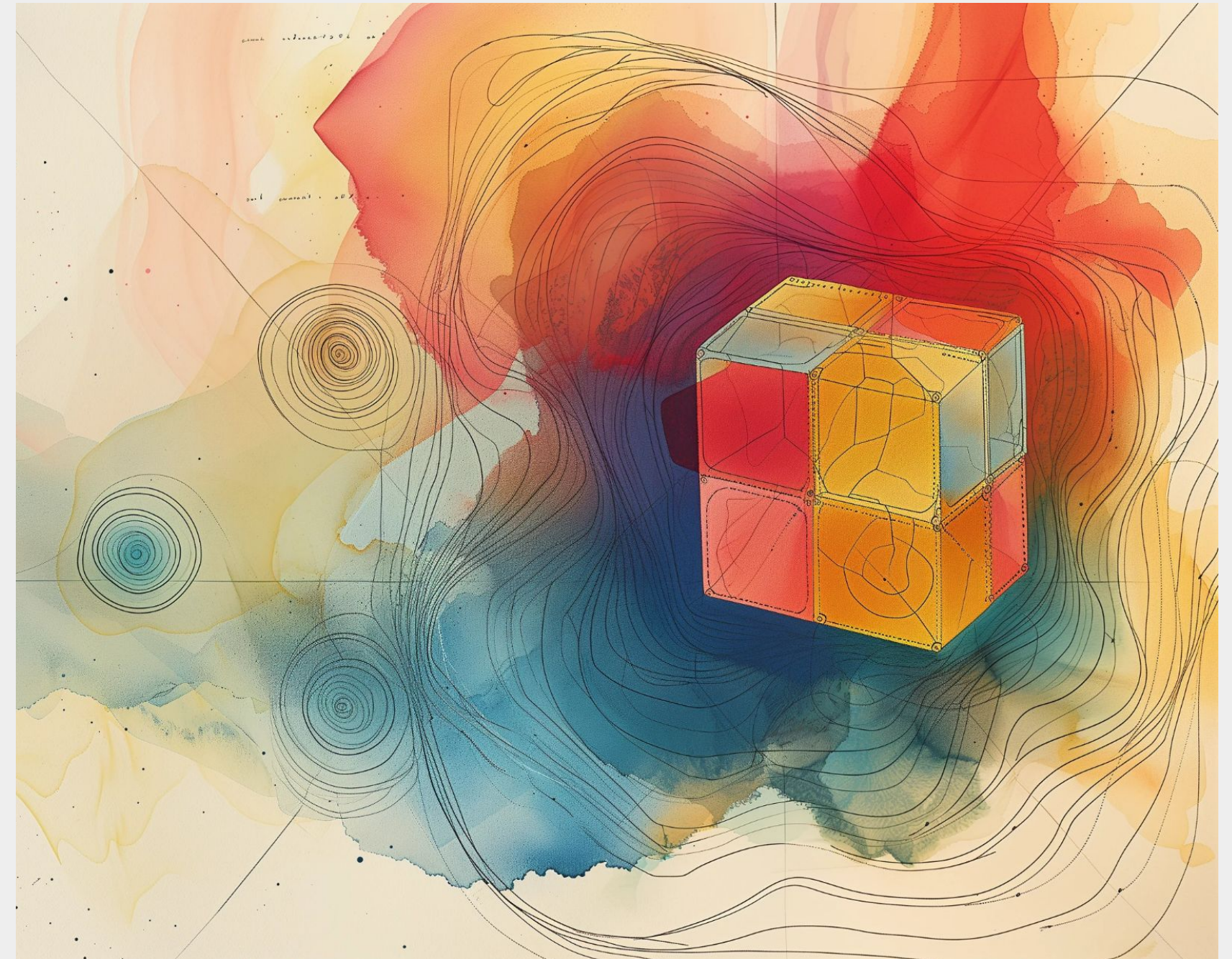
Compute Intensity

Operating directly in "Pixel Space" requires massive GPU power to calculate every individual pixel's noise.

Memory Demand

High-resolution images lead to an exponential increase in VRAM usage, often requiring industrial-grade servers.

Stable Diffusion



Stable diffusion - Latent space

Standard Diffusion Models (like early versions of OpenAI's DALL-E 2 or Google's Imagen) operate directly in the Pixel Space.

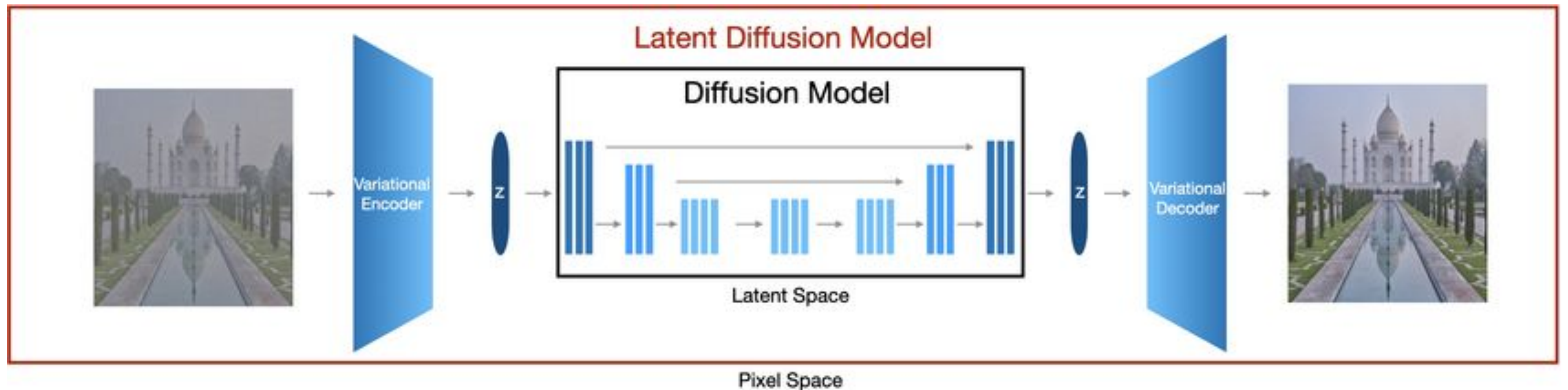
The Problem: High-resolution images contain millions of pixels. Calculating, adding, and removing noise for every single pixel across dozens of steps requires massive computing power and absurd amounts of VRAM.



Stable diffusion - Latent space

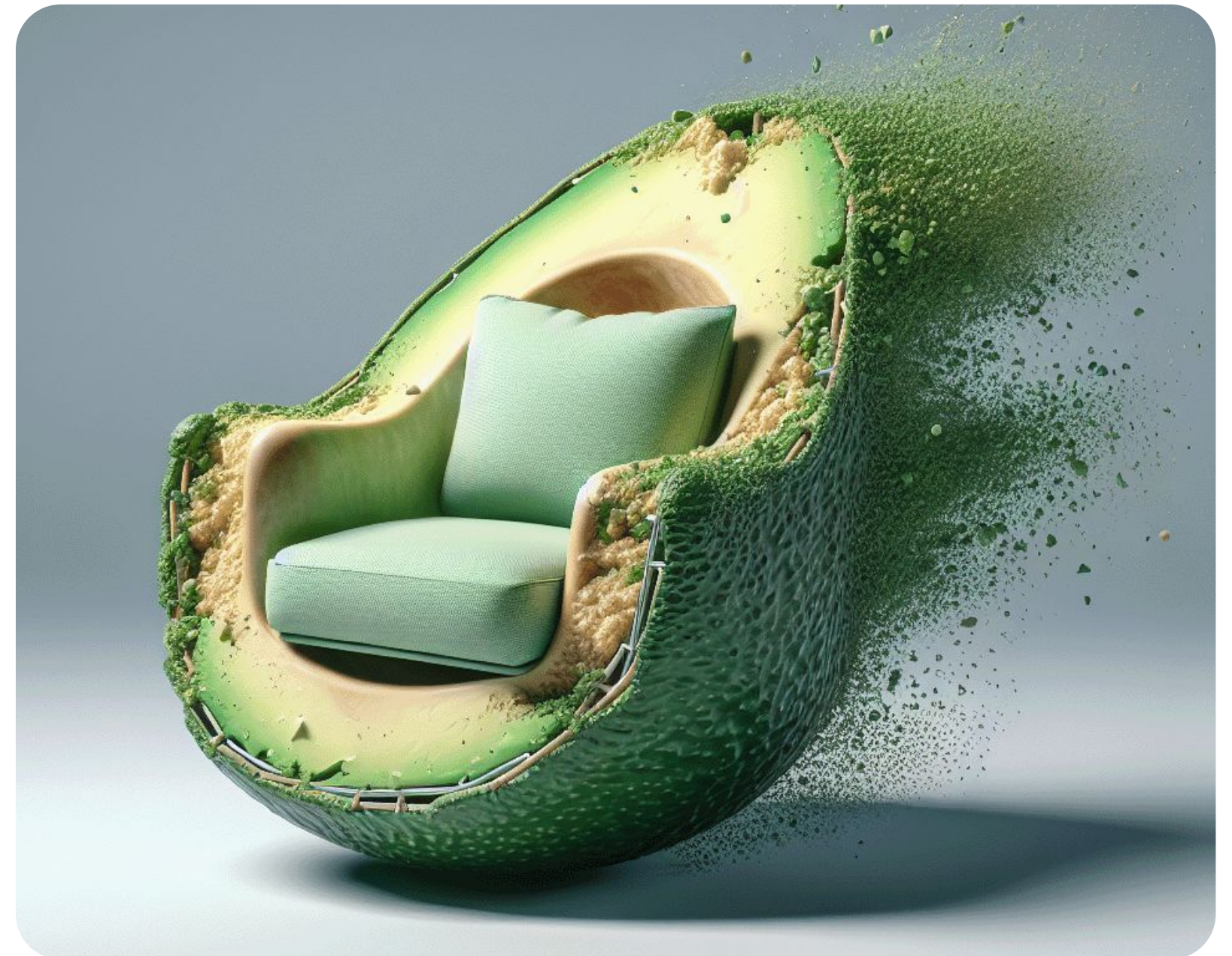
Stable Diffusion introduced a paradigm shift by working in **Latent Space** rather than Pixel Space. By using a Variational Autoencoder (VAE), images are compressed into a compact mathematical representation.

This allows the complex diffusion math to happen at a lower dimensionality, drastically reducing the required compute power and allowing AI generation on consumer hardware. An image of 512 x 512 pixels is compressed into a 64 x 64 latent matrix.



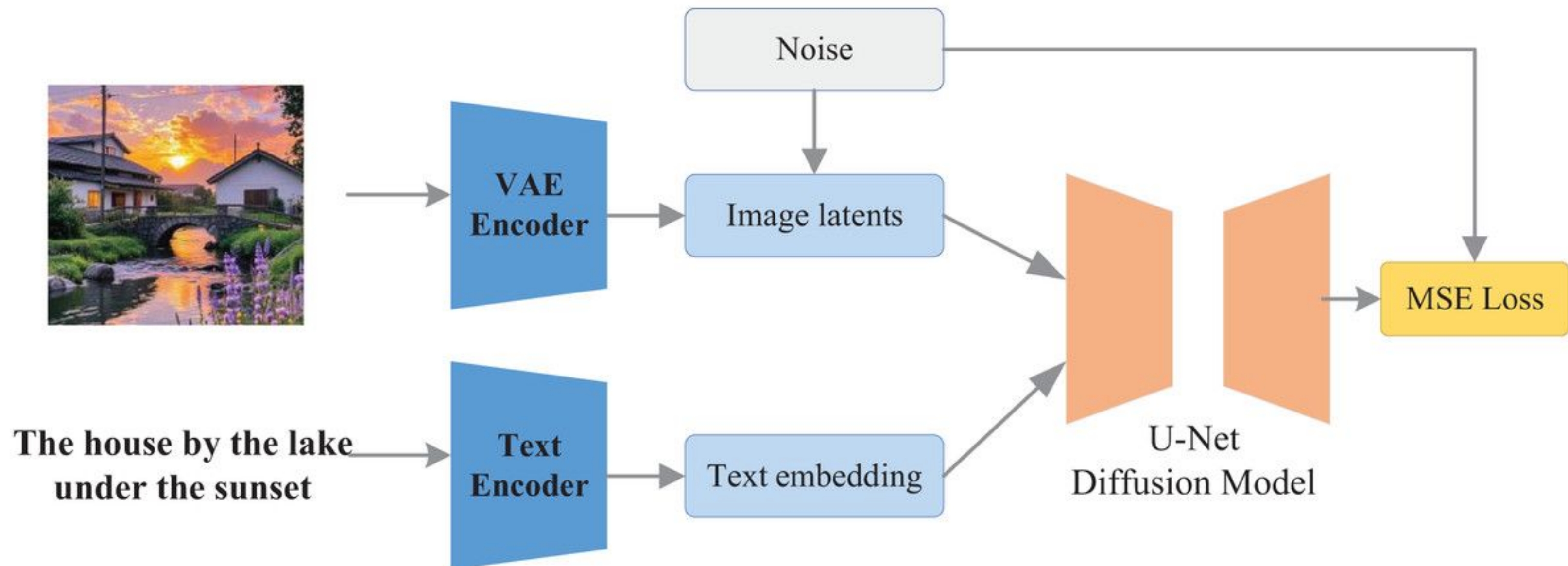
How text comes into play?

So far we have learned how to generate images but without any level of control of the output! How can we give a prompt to guide the model to generate an image?



Training with text: Conditioning

We make a conditioning (Text-to-Image) during the training process. To guide the image generation, we feed text prompts into the model using a text encoder (OpenCLIP). This forces the denoising process to align with your words.



Inside a diffusion model, we have two entirely different data types trying to collaborate:

The Spatial Data (U-Net): A grid of hidden visual features representing the noisy image.

The Contextual Data (Text Encoder): A sequence of text embeddings generated by a model like CLIP or T5.

Without cross-attention, the U-Net would generate a beautiful, crisp image - but it would be completely random because it has no way to inject the text's meaning into its spatial layers.

How Cross-Attention Works

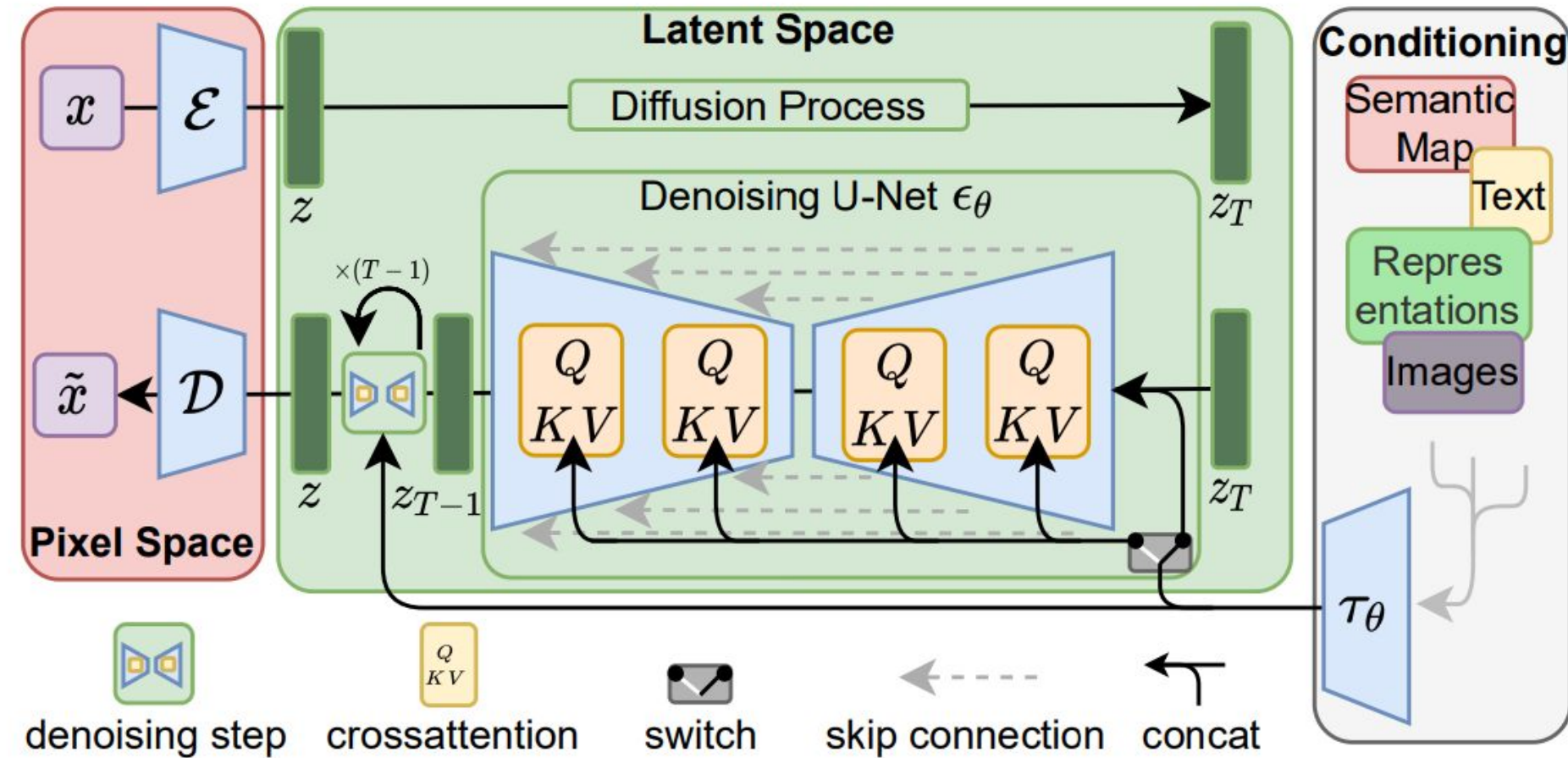
Cross-attention adapts the standard Transformer attention mechanism, but instead of looking at itself (self-attention), it looks across two different inputs. It uses three vectors: Queries (Q), Keys (K), and Values (V).

Queries (Q): Derived from the U-Net's spatial features. Every pixel-like location in the current U-Net layer asks: "What am I looking for?"

Keys (K): Derived from the text embeddings. Every word/token in your prompt says: "This is the concept I offer."

Values (V): Also derived from the text embeddings. Every word/token says: "This is the actual visual meaning to inject if you choose me."

How Cross-Attention Works



To combine them, the model projects the U-Net features to create Q , and the text features to create K and V . It then calculates the attention scores using the standard attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Step by step execution

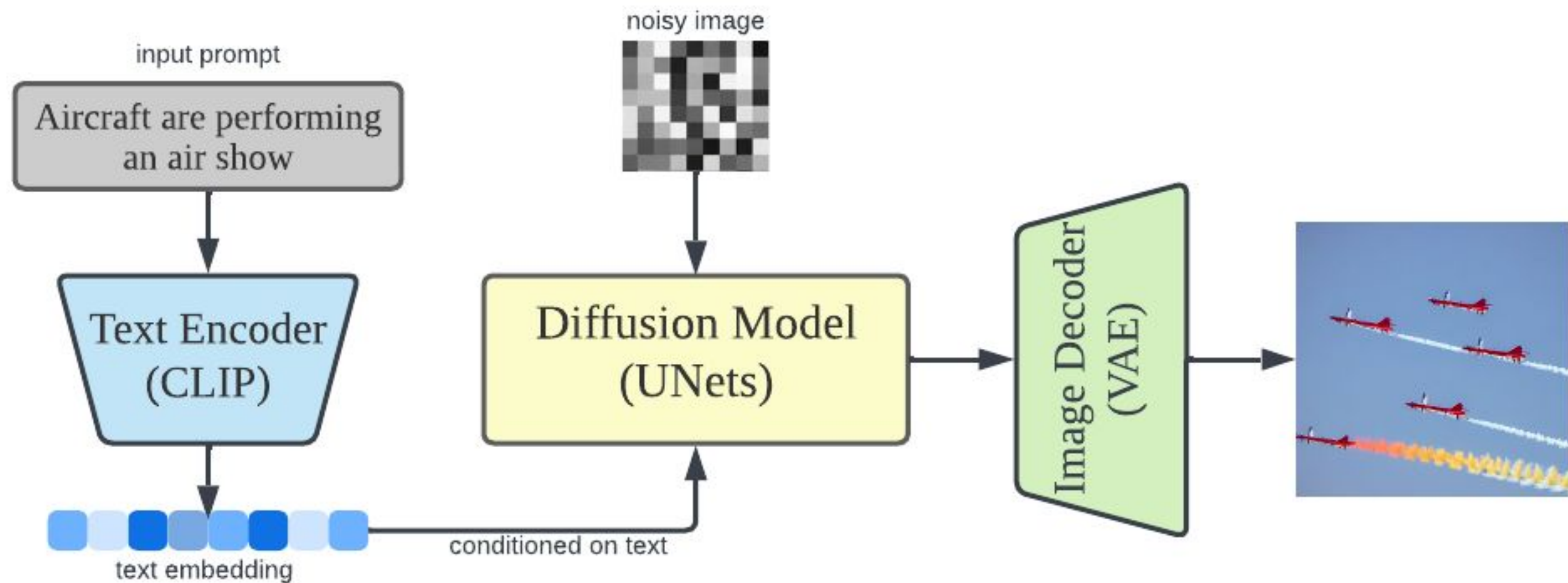
The Dot Product QK^T : The model multiplies every spatial location (Q) by every text token (K). This generates an Attention Map (a grid for every single word in your prompt). If we typed "a red apple," the "red" and "apple" tokens will yield high scores in the specific grid locations where the U-Net is starting to form the apple shape.

The Softmax: This normalizes the scores into probabilities (fractions between 0 and 1) so the U-Net knows exactly how much weight to give each word for that specific pixel area.

The Value Multiplication $\times V$: The U-Net multiplies these normalized weights by the text values (V). The final output is a text-conditioned feature map that is blended back into the U-Net's spatial layer.

Image generation

After the training we will use our model in combination with the same text encoder to produce new images.



Contrastive Language-Image Pretraining (CLIP)



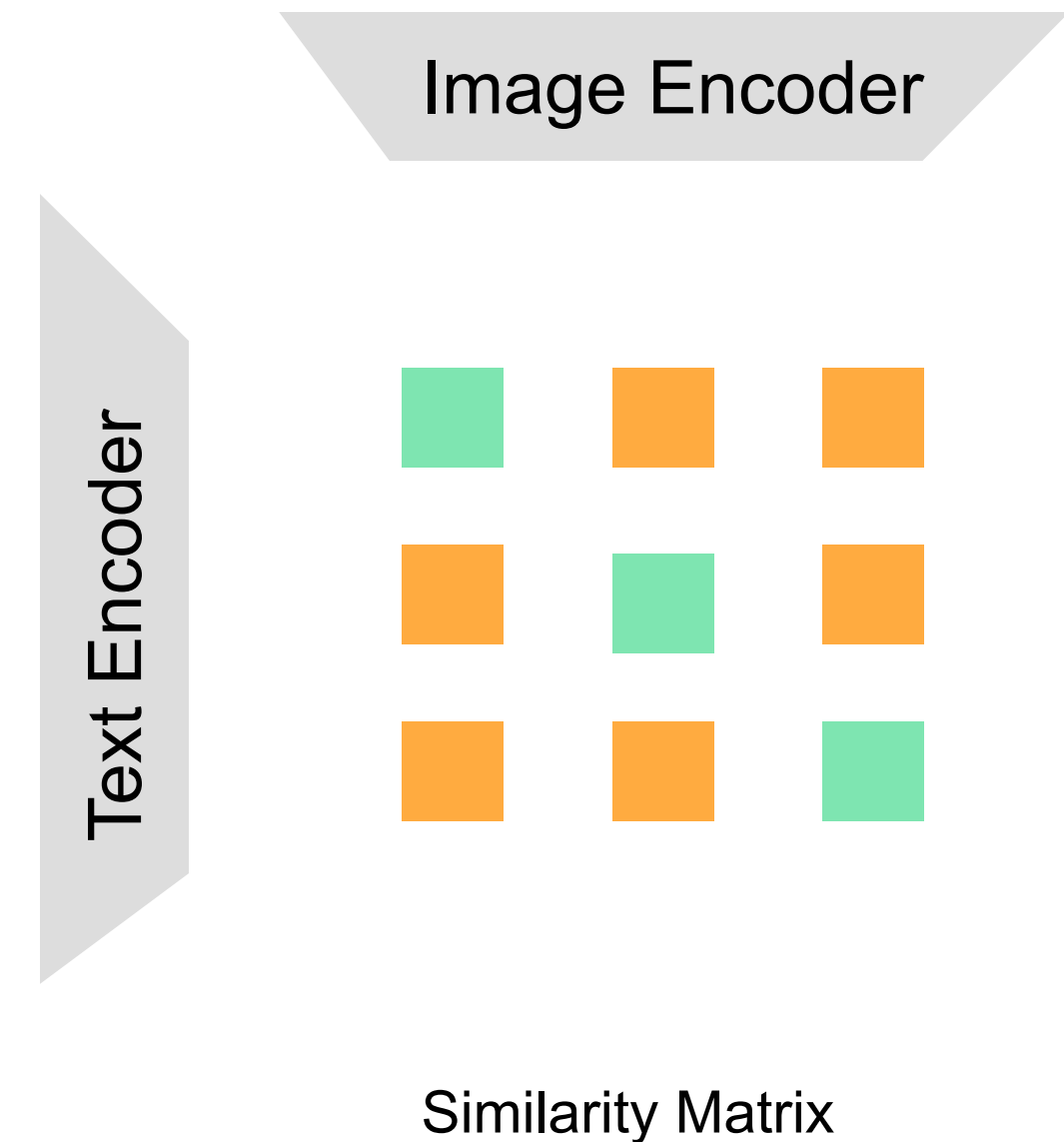
CLIP (Contrastive Language-Image Pre-training) is a neural network architecture developed by OpenAI that learns visual concepts from natural language supervision.

Unlike traditional models that use a fixed set of labels (e.g., ImageNet classes), CLIP is trained on 400 million image-text pairs collected from the internet, allowing it to understand the open world.

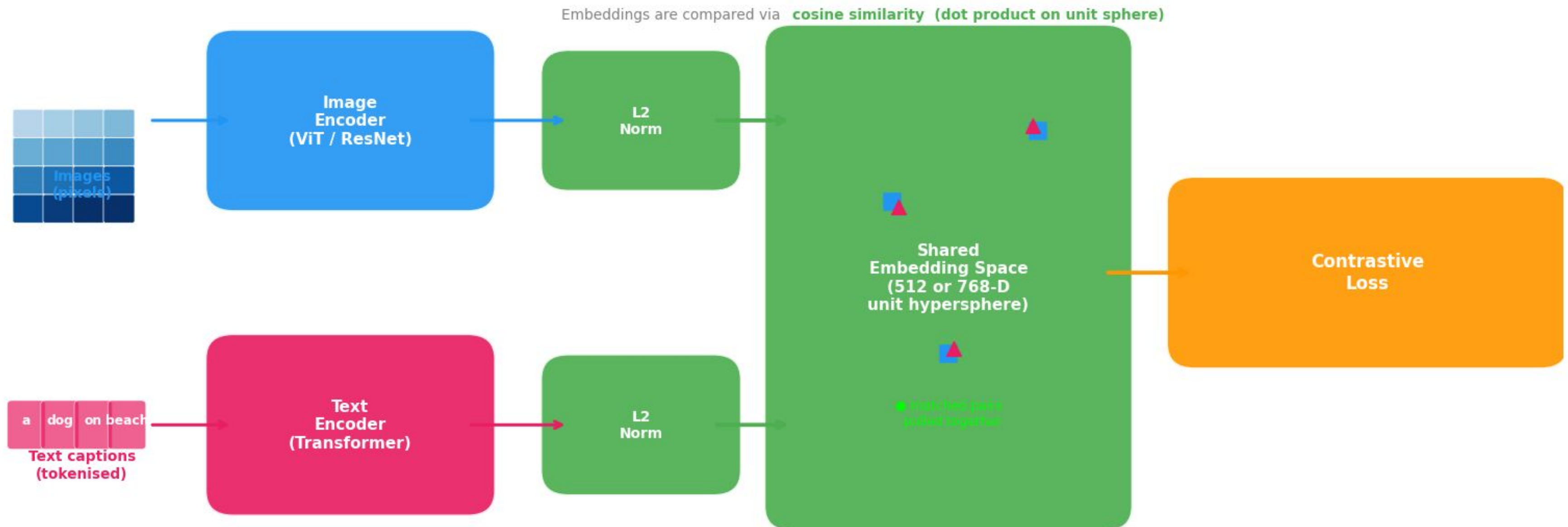
CLIP uses a dual-encoder design to process two different modalities simultaneously.

Image Encoder: Encodes pixels into vectors.

Text Encoder: Encodes sentences into vectors.



CLIP Architecture — Two Encoders, One Shared Space

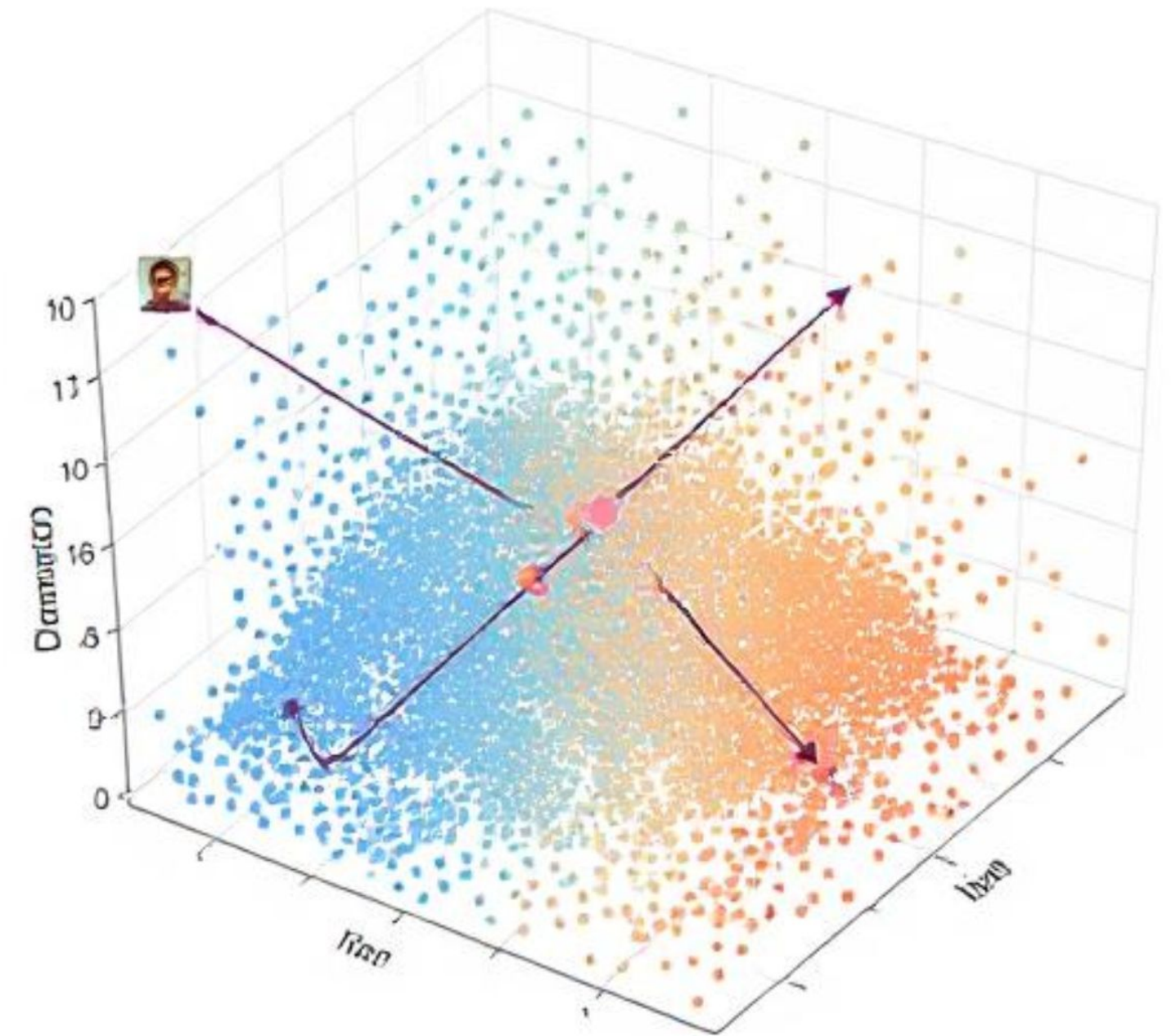


Linking Textual and Visual Semantics

First, all images and their associated captions are passed through their respective encoders, mapping all objects into an m-dimensional space.

Then, the cosine similarity of each (image, text) pair is computed.

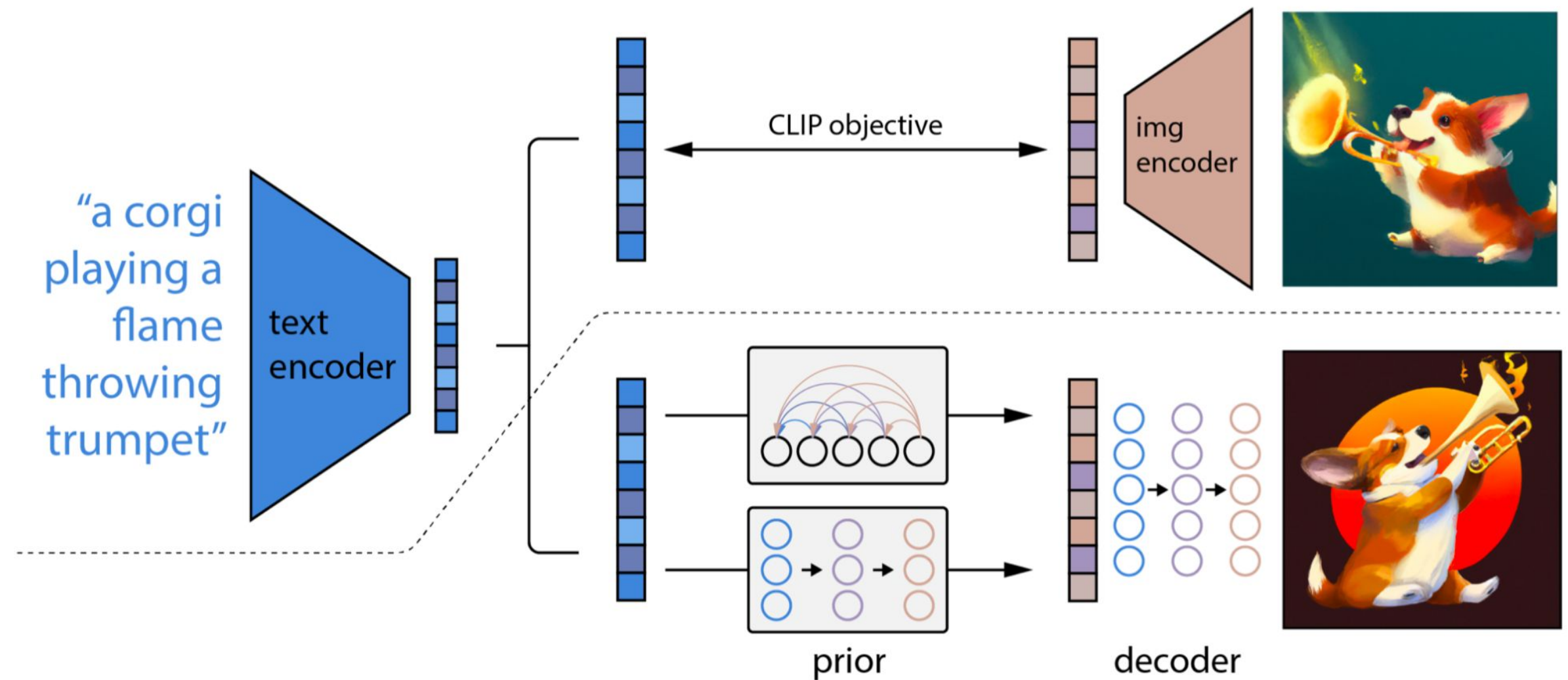
The training objective is to simultaneously **maximize** the cosine similarity between **correct** encoded image/caption pairs and **minimize** the cosine similarity between **incorrect** encoded image/caption pairs.



CLIP in diffusion training

Clip text embedding is fed to a diffusion model. This model generates an image embedding.

Embedding is used to condition a diffusion decoder producing the final image.



Thank you